

MWW Project-Page Dashboard Build Manual

June 2026

Contents

MWW Project-Page Dashboard	2
Build Manual	2
Table of contents	2
1. What you're building	3
2. Architecture at a glance	3
3. Prerequisites checklist	4
4. Phase 1 — Static site repository setup	4
4.1 Repo structure expected	5
4.2 Recommended preparation work	5
4.3 Create the dashboard-side clone	5
5. Phase 2 — External service accounts	5
5.1 Anthropic	5
5.2 Google Cloud (Maps, Geocoding, YouTube)	6
5.3 Telegram bot	6
5.4 GitHub	7
5.5 YouTube channel preparation	8
6. Phase 3 — VPS setup	8
6.1 Provision the VPS	8
6.2 Initial OS prep	8
6.3 Install Node.js 22	8
6.4 Install nginx + Certbot	9
6.5 Create the deploy user and directories	9
6.6 Clone the static-site repo	9
6.7 Configure git committer identity	9
6.8 Allow git to operate as root	9
7. Phase 4 — Domain + TLS	9
7.1 DNS	9
7.2 nginx config	10
7.3 TLS certificate	10
8. Phase 5 — Cloning the dashboard repo + per-company customization	11
8.1 Fork or copy this dashboard repo	11
8.2 Per-company customizations (do these before first run)	11
8.3 Install dependencies	12
9. Phase 6 — First-run configuration and bootstrap	12
9.1 Create the .env file	12
9.2 Build the cities → hub lookup	13
9.3 systemd service	13
9.4 First login & admin bootstrap	14
10. Phase 7 — One-time integration connects	14
10.1 YouTube connect	14
10.2 Per-hub Google review URLs	15

11. Phase 8 — cPanel side of the static site	15
11.1 Connect cPanel to the static-site repo	15
11.2 Configure auto-deploy via <code>.cpanel.yml</code>	15
11.3 The deploy click	15
12. Phase 9 — Daily backup setup	16
12.1 Create the backup repo on GitHub	16
12.2 Set up a separate deploy key for backups	16
12.3 Clone the backup repo on the VPS	16
12.4 The backup script	16
12.5 Cron job	16
13. Workflow reference: how everything connects in production	17
13.1 Submission flow	17
13.2 Publish flow	17
13.3 YouTube upload flow	17
13.4 Project page generation	18
13.5 Database schema	18
14. Customization points (per-company)	19
15. Common build-time gotchas	19
“REQUEST_DENIED” on the new-project address autocomplete	19
“fatal: detected dubious ownership in repository”	19
“non-fast-forward” push from the dashboard	19
YouTube uploads land on the wrong channel	19
Cannot GET <code>/<route></code> after deploying new code	20
413 Request Entity Too Large on video upload	20
Telegram returns “chat not found”	20
Backup script can’t push	20
Photos rotated sideways on iPhone	20
Bootstrap admin not created on first launch	20
The published page is missing the office-map iframe	20

MWW Project-Page Dashboard

Build Manual

A complete, step-by-step guide for building this app from scratch for a different company. Covers every external account, integration, server, deployment, and customization point. Read end-to-end before starting; the order of the steps matters because some integrations depend on artifacts created earlier.

Target audience: a developer comfortable with Linux, Node.js, Express, and basic DevOps (nginx, systemd, DNS, git). No prior familiarity with this codebase is assumed.

Table of contents

1. What you’re building
2. Architecture at a glance
3. Prerequisites checklist
4. Phase 1 — Static site repository setup
5. Phase 2 — External service accounts
 - 5.1 Anthropic
 - 5.2 Google Cloud (Maps, Geocoding, YouTube)
 - 5.3 Telegram bot
 - 5.4 GitHub

- 5.5 YouTube channel preparation
 - 6. [Phase 3 — VPS setup](#)
 - 7. [Phase 4 — Domain + TLS](#)
 - 8. [Phase 5 — Cloning the dashboard repo + per-company customization](#)
 - 9. [Phase 6 — First-run configuration and bootstrap](#)
 - 10. [Phase 7 — One-time integration connects](#)
 - 11. [Phase 8 — cPanel side of the static site](#)
 - 12. [Phase 9 — Daily backup setup](#)
 - 13. [Workflow reference: how everything connects in production](#)
 - 14. [Customization points \(per-company\)](#)
 - 15. [Common build-time gotchas](#)
-

1. What you're building

A Node.js / Express dashboard, deployed on its own VPS, that:

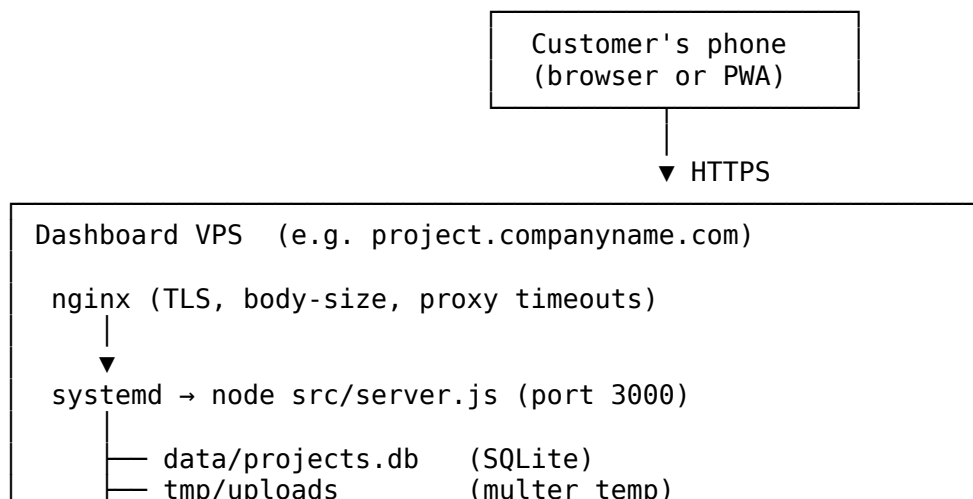
- Lets technicians submit completed-job reports from their phones.
- Generates SEO-optimized HTML project pages using an AI-written narrative and the company's voice guidelines.
- Writes those pages directly into the company's static-site git repo, patches related navigation / archive pages, updates the sitemap, commits, and pushes.
- Uploads videos to YouTube on behalf of the company.
- Generates ready-to-paste social media posts.
- Hands customers a Google-review QR code on the tech's phone screen.

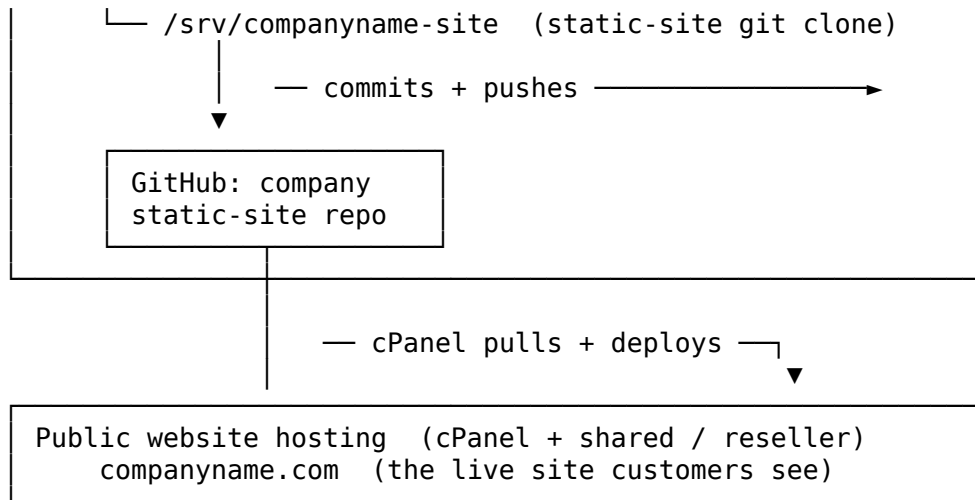
The dashboard never serves the public site. It only edits the static site's git repository. A separate hosting setup (cPanel + Namecheap hosting in the MWW case) serves the public site by pulling that repo.

This document is your roadmap to recreating that for a different company. The companies need to share the same model: **multi-hub local service business with a static-site main website**.

If the company's website is a Wordpress site, a Shopify site, or a SaaS platform, the publishing layer (src/routes/publish.js, src/lib/spoke-update.js, etc.) needs to be rewritten to talk to that CMS's API. The core form / AI / role-management code stays the same.

2. Architecture at a glance





External services the dashboard talks to:

- Anthropic API — narrative, FAQ, video metadata, social-post captions
- Google Places API — address autocomplete
- Google Geocoding API — server-side scripts
- Telegram Bot API — admin approval notifications
- YouTube Data API v3 — video uploads
- GitHub — static-site repo + dashboard repo + backup repo

3. Prerequisites checklist

Before starting, gather or set up:

- ☐ A VPS — Ubuntu 22.04 LTS or 24.04 LTS, 2GB RAM minimum, 4GB recommended. (The MWW deployment is on IONOS but any provider works: Linode, DigitalOcean, Hetzner, etc.)
- ☐ A domain you control. We'll use `project.companyname.com` as the example throughout this doc.
- ☐ An SSH keypair on your local machine (for connecting to the VPS).
- ☐ A GitHub account with permission to create repos.
- ☐ A Google account that will own the company's YouTube brand channel and Google Business Profile listings.
- ☐ An Anthropic account with billing set up.
- ☐ A Telegram account for the admin who'll receive approval pings.
- ☐ The company's brand assets: logo (PNG, ideally 512×512+), brand colors, address(es) for each hub office, phone numbers per hub.
- ☐ The company's static-site source code — either an existing repo or knowledge of how the site is structured (we'll mirror it).
- ☐ A list of "service-area cities" the company serves. We use these to build the dashboard's address autocomplete biasing.
- ☐ List of services the company offers (e.g. for a cleaning company: window cleaning, gutter cleaning, power washing). We build one form per service.

4. Phase 1 — Static site repository setup

The dashboard exists to commit to a static-site repo. We need that repo to be in good shape first.

4.1 Repo structure expected

The dashboard expects the static-site repo to look approximately like this:

```
companyname-site/
├── projects/                ← we write into here
│   ├── img/                ← project photos go here (.webp)
│   ├── index.html          ← master archive (we regenerate)
│   ├── window-cleaning.html ← per-service archive (we regenerate)
│   ├── gutter-cleaning.html
│   └── ...
├── images/team/            ← tech avatars go here
├── sitemap.xml              ← we add/refresh <url> entries
├── chicago.html             ← spoke pages (per-hub locations)
├── lisle.html
├── ...
└── (everything else)
```

A “spoke page” is a city / location page (e.g. Northbrook, Naperville) that already exists on the company site. Each spoke page has a “Recent Projects” section with up to N tiles. The dashboard updates those tiles when a new project is published.

Each spoke page should already include:

- A “Recent Projects” section with a known CSS selector containing 3-6 .project-card slots (initially “Coming soon” placeholders).
- A footer with a “View all N projects in [City]” link.

4.2 Recommended preparation work

If your company doesn’t already have a static-site setup like this, do the following first:

1. Set up the main domain hosting (cPanel + a static site, or GitHub Pages, or Netlify) and point it at a git repository.
2. Build hub pages: one per office location, with the structure above. The dashboard will hook into them automatically as long as you tell it (via scripts/build-cities.js) which spoke pages belong to which hub.
3. Decide on a slug pattern for project pages. The default in this codebase is [service]-in-[city]-[mm]-[dd]-[yy] — e.g. window-cleaning-in-naperville-06-12-26. You can change this in src/lib/slug.js → buildSlug().

4.3 Create the dashboard-side clone

The VPS needs a local clone of the static-site repo that the dashboard can write to and git push from. We’ll do this in Phase 6 when the VPS is set up.

5. Phase 2 — External service accounts

Do this **before** touching the VPS. Several integrations need API credentials that go in the .env file.

5.1 Anthropic

Used for: narrative, FAQ, video metadata, social-post captions.

1. Go to <https://console.anthropic.com> and create an account.
2. Set up billing — projects-day-1 usage is tiny (each narrative regeneration is ~\$0.01-0.02 with prompt caching), but you need a credit card on file.

3. **Create an API key.** Name it something like `mwv-dashboard-prod`. Copy it; you won't be able to see it again.
4. Choose a model. Default in the code is `claude-sonnet-4-6`. For higher-quality narratives, switch to `claude-opus-4-8` (more expensive). For lowest cost, switch to `claude-haiku-4-5-20251001`.
5. Save the API key + the model identifier — you'll paste them into `ANTHROPIC_API_KEY` and `ANTHROPIC_MODEL` in `.env`.

5.2 Google Cloud (Maps, Geocoding, YouTube)

Used for: address autocomplete on the form, server-side geocoding, YouTube uploads.

1. Go to <https://console.cloud.google.com> and create a project (e.g. `companyname-dashboard`).
2. Enable these APIs (APIs & Services → Library, search and Enable each):
 - **Maps JavaScript API**
 - **Places API (New)** — for the autocomplete library
 - **Geocoding API**
 - **YouTube Data API v3**
3. **Create two API keys** (Credentials → Create credentials → API key):

Key A: browser-restricted (Maps + Places)

 - Restrict by HTTP referrer
 - Add `https://project.companyname.com/*` (and during local dev, `http://localhost:*`)
 - Limit to: Maps JavaScript API, Places API (New)
 - Save as `GOOGLE_MAPS_API_KEY`

Key B: IP-restricted (Geocoding)

 - Restrict by IP address
 - Add the VPS's public IP
 - Limit to: Geocoding API
 - Save as `GOOGLE_GEOCODING_API_KEY`
 - Why a second key: Geocoding API ignores `Referer` headers from server-side requests, so a browser-restricted key throws `REQUEST_DENIED`. IP restriction is the right model.
4. **Configure OAuth consent screen** (for YouTube uploads):
 - APIs & Services → OAuth consent screen
 - User type: **External**
 - App name: `companyname Dashboard`
 - Support email + developer email: the admin's Gmail
 - Add the YouTube Data API v3 scope `youtube.upload`
 - Click **Publish app** so the OAuth grant doesn't expire after 7 days. Since the only scope is "upload to your own channel," no verification is required by Google.
5. **Create the OAuth 2.0 Client ID:**
 - Credentials → Create credentials → OAuth client ID
 - Type: **Web application**
 - Name: `MWV Dashboard YouTube Client`
 - Authorized redirect URI: `https://project.companyname.com/youtube/callback`
 - Click Create
 - Copy the **Client ID** and **Client Secret**
 - Save as `YOUTUBE_CLIENT_ID` and `YOUTUBE_CLIENT_SECRET`

5.3 Telegram bot

Used for: admin approval notifications when a tech submits a project.

1. On Telegram, message @BotFather.
2. Send /newbot. Pick a name (e.g. My Window Washing Dashboard) and a username ending in bot (e.g. mwwdashboard_bot).
3. BotFather returns a token — looks like 123456789:AAExxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx. Save it.
4. **Create a Telegram group** with all the admins who should receive pings.
5. Add the bot to the group.
6. Have someone send any message in the group (the bot can only see messages after at least one human message exists).
7. To find the chat ID:
 - In a browser, visit <https://api.telegram.org/bot<TOKEN>/getUpdates>
 - Find the JSON entry for your group; the chat.id is a negative number like -1001234567890.
8. Save the bot token + chat ID as TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID.

5.4 GitHub

You'll need three repos:

- **The static-site repo** (probably already exists)
- **The dashboard app repo** (this codebase, forked or new)
- **A private backup repo** (for daily SQLite backups)

For each:

1. Create the repo on GitHub.
2. On the VPS (Phase 6), create an SSH key per repo:


```
ssh-keygen -t ed25519 -C "vps-deploy-companyname-site" -f ~/.ssh/companyname-site
ssh-keygen -t ed25519 -C "vps-deploy-companyname-dashboard" -f ~/.ssh/companyname-dashboard
ssh-keygen -t ed25519 -C "vps-deploy-companyname-backup" -f ~/.ssh/companyname-backup
```
3. Add each public key (.pub file) as a **Deploy Key** on the matching GitHub repo, with write access.
4. Configure ~/.ssh/config so each repo uses its own key:


```
Host github-site
  HostName github.com
  User git
  IdentityFile ~/.ssh/companyname-site
Host github-dashboard
  HostName github.com
  User git
  IdentityFile ~/.ssh/companyname-dashboard
Host github-backup
  HostName github.com
  User git
  IdentityFile ~/.ssh/companyname-backup
```
5. Test each: `ssh -T git@github-site` etc. should print "Hi ! You've successfully authenticated...".

Why one key per repo? GitHub doesn't let the same deploy key be attached to multiple repos. The SSH config aliases let us route each repo's clone URL to the right key.

5.5 YouTube channel preparation

The dashboard uploads to a single YouTube channel. If the company doesn't already have a YouTube channel, create one **as a brand account** (not as a personal channel):

1. Sign in to <https://www.youtube.com> with the Gmail that will own the channel.
2. Click your avatar → **Create a channel**.
3. Choose **Use a custom name** so the channel can have its own identity (e.g. "My Window Washing").
4. Once created, verify at <https://myaccount.google.com/brandaccounts> that it shows up as a Brand Account owned by your Gmail.

A brand account is preferred over a personal channel because it lets multiple humans manage the channel without sharing a password — but the dashboard's OAuth flow only ever connects one channel.

If the company already has a YouTube channel that's a personal channel (not brand), you can still use it, but reading the connection steps in section 10.1 of the User Manual carefully is important — the brand-account chooser screen in OAuth is critical to landing uploads on the right channel.

6. Phase 3 — VPS setup

6.1 Provision the VPS

Pick a provider, spin up an Ubuntu 22.04 / 24.04 instance:

- 2 GB RAM minimum (4 GB if you're doing heavy YouTube uploads)
- 20 GB SSD
- Root SSH access

For the rest of this section we'll assume:

- VPS public IP: 203.0.113.42
- You've added your local SSH key to `/root/.ssh/authorized_keys`

6.2 Initial OS prep

SSH in as root and run:

```
apt update && apt upgrade -y
apt install -y curl git build-essential ufw fail2ban
```

```
# Firewall: allow SSH, HTTP, HTTPS only
ufw allow OpenSSH
ufw allow 'Nginx Full' # we'll install nginx next
ufw --force enable
```

```
# Timezone (adjust to your office)
timedatectl set-timezone America/Chicago
```

```
# Hostname
hostnamectl set-hostname dashboard-vps
```

6.3 Install Node.js 22

Use NodeSource to get the latest LTS:


```
curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
apt install -y nodejs
node -v    # should print v22.x
```

6.4 Install nginx + Certbot

```
apt install -y nginx certbot python3-certbot-nginx
systemctl enable --now nginx
```

6.5 Create the deploy user and directories

We'll deploy as root for simplicity (the MWW setup uses root). For a hardened production setup, create a non-root user and adjust permissions accordingly.

```
mkdir -p /var/www/companyname-dashboard-app
mkdir -p /srv/companyname-site
```

6.6 Clone the static-site repo

```
cd /srv
git clone github-site:org/companyname-site.git companyname-site
```

(Note the github-site: prefix — that's the SSH config alias from step 5.4. The trailing :org/companyname-site.git is the GitHub user/repo path.)

6.7 Configure git committer identity

```
cd /srv/companyname-site
git config user.name "MWW Dashboard"
git config user.email "dashboard@companyname.com"
```

(These can be overridden by the .env later, but setting them here covers manual debugging git operations.)

6.8 Allow git to operate as root

```
git config --global --add safe.directory /srv/companyname-site
git config --global --add safe.directory /var/www/companyname-dashboard-app
```

7. Phase 4 — Domain + TLS

7.1 DNS

In your domain registrar, create an A record:

project.companyname.com → 203.0.113.42 (the VPS IP)

Optionally a wildcard / www. variant:

www.project.companyname.com → 203.0.113.42

Wait for DNS propagation (dig +short project.companyname.com should return the VPS IP).

7.2 nginx config

Create /etc/nginx/sites-available/project.companyname.com:

```
server {
    listen 80;
    listen [::]:80;
    server_name project.companyname.com www.project.companyname.com;

    # Certbot will manage the HTTPS server block below.
    location / {
        return 301 https://$host$request_uri;
    }
}

server {
    server_name project.companyname.com www.project.companyname.com;

    # Allow up to 2 GB body for video uploads.
    client_max_body_size 2G;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host                $host;
        proxy_set_header X-Real-IP            $remote_addr;
        proxy_set_header X-Forwarded-For      $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto    $scheme;
        proxy_set_header Connection           "";

        # Streaming for big uploads – don't buffer the whole body in nginx.
        proxy_request_buffering off;

        proxy_connect_timeout 30s;
        proxy_send_timeout    3600s;
        proxy_read_timeout     3600s;
    }

    listen 443 ssl;
    listen [::]:443 ssl;
    # Certbot will fill the ssl_certificate lines below.
}
```

Enable + test:

```
ln -s /etc/nginx/sites-available/project.companyname.com /etc/nginx/sites-enabled/
nginx -t
systemctl reload nginx
```

7.3 TLS certificate

```
certbot --nginx -d project.companyname.com -d www.project.companyname.com
```

Certbot interactively asks for your email, accepts the TOS, and modifies the nginx config to add `ssl_certificate` lines pointing at the new certificate. It also sets up auto-renewal via systemd timer.

Verify:

```
curl -I https://project.companyname.com # should hit the dashboard
```

8. Phase 5 — Cloning the dashboard repo + per-company customization

8.1 Fork or copy this dashboard repo

```
cd /var/www
git clone github-dashboards:org/companyname-dashboard-app.git mww-dashboard-app
cd mww-dashboard-app
```

(Replace the GitHub path with your fork of the MWW dashboard repo, or push a copy of this codebase to a new private repo.)

8.2 Per-company customizations (do these before first run)

The codebase has several places that bake in MWW-specific values. Change each to match the target company.

a. `src/lib/slug.js` — services

The SERVICES array at the top defines which services the company offers. Each service has:

- `value` — URL-safe identifier (e.g. power-washing)
- `label` — human-readable name
- `schemaType` — schema.org Service type
- `spokeTag` — text shown on the spoke-page “Recent Projects” tile
- `servicePage` — the matching service page on the main site
- `iconSvg` — inline SVG for the service picker

Add / remove services here. For each service, you also need:

- A form partial in `src/views/partials/details-<value>.handlebars` (copy + modify one of the existing three).
- An entry in `DETAILS_PARTIALS` in `src/routes/projects.js` mapping the service value to the partial name.
- An entry in `CHALLENGE_CHOICES` and `BULLET_PLACEHOLDER` in `src/routes/projects.js` for that service.
- A scope-tag builder in `src/lib/render.js` (look for `windowScopeTags`, `gutterScopeTags`, `powerScopeTags` and add a new one).
- Service-specific extras handling in `collectExtras()` in `src/routes/projects.js`.

b. `scripts/build-cities.js` — hubs

The HUBS object lists every office location the company has. Per hub:

- `hubSlug` — URL fragment / identifier
- `name` — display name
- `phone`, `phoneDigits` — for the page footer + JSON-LD
- `address`, `addressShort`, `addressCity`
- `parentOrgUrl` — the canonical hub page on the main site, with `#localbusiness` anchor (so JSON-LD identifies the parent)
- `mapEmbedQuery` — what to send to Google Maps Embed when a hub’s GBP iframe isn’t available
- `lat`, `lng` — for nearest-hub binding
- `gbpEmbedUrl` — optional, paste the iframe src from the hub’s Google Business Profile “Share → Embed a map” dialog

Edit the HUBS, then run `npm run build-cities` (in Phase 6) to generate `data/cities.json`.

c. `src/lib/render.js` — site root + constants

- SITE_ROOT (line 17) — change from `https://www.mywindowwashing.com` to `https://www.companyname.c`
- TRUST_SIGNALS — the four trust badges shown on every project page (“Insured”, “Family Owned”, “5 Reviews”, “Since 2003”). Edit per company.
- Phone fallback in `lib/video-meta.js` defaults to MWW’s Northbrook number — change to whichever hub is the company’s main number.

d. templates/project-page.hbs — the page template

This is the biggest template, ~880 lines. The brand-specific bits:

- Hero / nav: company logo path, header phone
- Color palette: search for hex codes like `#133047` (navy), `#4caf50` (green CTA), and globally replace per brand
- Footer: company address, social links, copyright text
- Footer “About us” + “Service areas” links — point at the company site’s nav

e. public/manifest.json + icons

Replace `icon-*.png` and `apple-touch-icon.png` with the company’s logo. Adjust `name`, `short_name`, `description`, `theme_color`, `background_color`. Then update the `<link rel="manifest">` and related meta tags in `src/views/layouts/main.handlebars` if you changed icon paths.

f. src/lib/narrative.js — voice prompt

The system prompt is heavily voice-tuned for MWW (calm, craftsman, no praise words, etc.). For a different company, edit:

- Company name + region in the first paragraph
- The example narrative at the end (the “Highland Park ravine homes” one) — write a fresh example in the company’s actual voice
- Forbidden words / mandatory style rules

g. src/lib/faq.js — FAQ prompt

Similar — update the company name + phone fallback.

h. src/lib/video-meta.js + src/lib/social-post.js

Same — update company name and brand voice.

i. data/cities.json

Regenerated by `npm run build-cities` from the HUBS list + a scrape of the spoke pages. Don’t hand-edit; rerun the script after changes.

8.3 Install dependencies

```
cd /var/www/companyname-dashboard-app
npm install
```

This installs Express, better-sqlite3, sharp, heic-convert, cheerio, simple-git, the Anthropic SDK, express-handlebars, multer, dotenv, etc. About 80 MB on disk.

9. Phase 6 — First-run configuration and bootstrap

9.1 Create the .env file

```
cd /var/www/companyname-dashboard-app
cp .env.example .env
nano .env
```

Fill in every value. Reference:

```

# Auth + sessions
DASHBOARD_PASSWORD=ChooseAnInitialPasswordForTheAdmin # used once to bootstrap
SESSION_SECRET=GenerateALongRandomStringForSigning # openssl rand -hex 32

# HTTP
PORT=3000

# Anthropic
ANTHROPIC_API_KEY=sk-ant-...
ANTHROPIC_MODEL=claude-sonnet-4-6

# Google APIs
GOOGLE_MAPS_API_KEY=AIzaSy... # browser-restricted
GOOGLE_GEOCODING_API_KEY=AIzaSy... # IP-restricted

# Static-site repo
SITE_REPO_PATH=/srv/companyname-site
SITE_REPO_BRANCH=master # or main, depending on the repo
SITE_REPO_PUSH=true

# Git identity for commits
GIT_AUTHOR_NAME=Companyname Dashboard
GIT_AUTHOR_EMAIL=dashboard@companyname.com

# Telegram approval ping
TELEGRAM_BOT_TOKEN=123456789:AAE...
TELEGRAM_CHAT_ID=-1001234567890
DASHBOARD_BASE_URL=https://project.companyname.com

# YouTube upload
YOUTUBE_CLIENT_ID=...apps.googleusercontent.com
YOUTUBE_CLIENT_SECRET=GOCSPX-...
YOUTUBE_REDIRECT_URI=https://project.companyname.com/youtube/callback
YOUTUBE_DEFAULT_PRIVACY=public

Save (Ctrl+O, Enter, Ctrl+X).

```

9.2 Build the cities → hub lookup

```

cd /var/www/companyname-dashboard-app
npm run build-cities

```

This scans every spoke page (<hub>.html) in the static-site repo, detects which hub each page belongs to by looking at its links to a hub identifier, geocodes any cities listed in spoke pages, and writes data/cities.json.

If a spoke page can't be auto-detected, the script logs it as unmapped and you'll need to add a manual entry. Commit data/cities.json after a successful build:

```

cd /var/www/companyname-dashboard-app
git add data/cities.json && git commit -m "Build cities for companyname"
git push

```

9.3 systemd service

Create /etc/systemd/system/companyname-dashboard.service:

```
[Unit]
```

Description=Companyname Project-Page Dashboard
After=network.target

```
[Service]
Type=simple
User=root
WorkingDirectory=/var/www/companyname-dashboard-app
ExecStart=/usr/bin/node src/server.js
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal
```

```
[Install]
WantedBy=multi-user.target
```

Enable and start:

```
systemctl daemon-reload
systemctl enable companyname-dashboard
systemctl start companyname-dashboard
systemctl status companyname-dashboard
journalctl -u companyname-dashboard -n 50 --no-pager
```

You should see:

Companyname dashboard listening on http://localhost:3000

Then in a browser, hit https://project.companyname.com. You should land on the login page.

9.4 First login & admin bootstrap

On the **first request** with an empty users table, the server logs:

```
[auth] users table empty + DASHBOARD_PASSWORD set — bootstrapping admin user 'admin'.
```

Log in as admin with the DASHBOARD_PASSWORD you set in .env.

Immediately: 1. Go to **Team** in the nav. 2. Click your own row → reset password to something stronger. 3. Add real team members (techs and any other admins). 4. After everything's set up, you can remove or rotate the bootstrap admin.

10. Phase 7 — One-time integration connects

10.1 YouTube connect

On the dashboard, signed in as admin:

1. Open **Videos** in the top nav.
2. Click ► **Connect YouTube channel**.
3. Sign in with the Gmail that owns the company brand account.
4. Through the “Google hasn’t verified this app” warning: **Advanced → Go to Companyname Dashboard (unsafe)**.
5. On the brand-account chooser, pick the company channel (not your personal channel).
6. Allow.
7. You should land back on /videos with a green ● Connected banner.

Verify by uploading a 5-second test clip with visibility = Unlisted, then check the upload landed on the right channel at <https://studio.youtube.com>. Delete the test after.

10.2 Per-hub Google review URLs

Signed in as admin:

1. Open **Review** in the top nav.
 2. Click **** Manage hub review URLs****.
 3. For each hub, get the Google review short link from <https://business.google.com> → switch to that location → **Read reviews** → **Get more reviews**. Paste the link.
 4. **Save all**.
-

11. Phase 8 — cPanel side of the static site

This is the public hosting side, separate from the VPS.

11.1 Connect cPanel to the static-site repo

1. cPanel → **Git Version Control** → **Create**
2. Repository URL: the GitHub `git@github.com:org/companyname-site.git` URL (you'll need a deploy key here too — cPanel generates one for you in the same screen; copy it as a deploy key on GitHub with read access).
3. Repository path: `/home/<cpanel-user>/companyname-site`
4. Repository name: `companyname-site`
5. Branch: `master`
6. Click **Create**.

11.2 Configure auto-deploy via `.cpanel.yml`

Add a `.cpanel.yml` file to the static-site repo root:

```
---
deployment:
  tasks:
    - export DEPLOYPATH=/home/<cpanel-user>/public_html
    - /bin/cp -R * $DEPLOYPATH
```

This tells cPanel where to copy the files when you click “Deploy HEAD Commit”. Commit and push.

11.3 The deploy click

After every push from the dashboard:

1. cPanel → **Git Version Control** → **Manage**
2. Click **Update from Remote** (pulls the new commit into the local clone)
3. Click **Deploy HEAD Commit** (runs `.cpanel.yml`)

This is the “cPanel deploy step” referenced throughout the User Manual. You can do it batched at end-of-day across multiple publishes.

12. Phase 9 — Daily backup setup

The SQLite database holds all submitted projects, drafts, users, and runtime settings (YouTube refresh token, hub review URLs). Backing it up nightly to a private GitHub repo protects against VPS loss.

12.1 Create the backup repo on GitHub

Private repo named e.g. `companyname-dashboard-backups`.

12.2 Set up a separate deploy key for backups

(Same SSH config alias pattern from section 5.4 — see `github-backup` host.)

12.3 Clone the backup repo on the VPS

```
cd /root
git clone github-backup:org/companyname-dashboard-backups.git
```

12.4 The backup script

The repo already includes `scripts/backup-db.sh`. Review it:

```
#!/bin/bash
set -euo pipefail

DB=/var/www/companyname-dashboard-app/data/projects.db
REPO=/root/companyname-dashboard-backups
STAMP=$(date +%F)

cd "$REPO"
mkdir -p backups

sqlite3 "$DB" .dump > "backups/projects-$STAMP.sql"

git add backups/
if ! git diff --cached --quiet; then
    git commit -m "Backup $STAMP"
    for i in 1 2 4 8; do
        git push && exit 0
        sleep "$i"
    done
    echo "Backup push failed after retries" >&2
    exit 1
fi
```

Make it executable + edit the paths to match your install:

```
chmod +x /var/www/companyname-dashboard-app/scripts/backup-db.sh
```

12.5 Cron job

```
crontab -e
```

Add:

```
0 3 * * * /var/www/companyname-dashboard-app/scripts/backup-db.sh >> /var/log/dashboard-backup.log 2>&1
```


This runs the backup at 3:00 AM daily. Check the log next morning.

13. Workflow reference: how everything connects in production

This section describes what happens when a tech submits a project, end to end, so you can debug and modify the flow with confidence.

13.1 Submission flow

1. Tech opens /new → picks service → fills form → hits Save & Send.
2. Browser POSTs to /new (multipart/form-data).
3. `src/routes/projects.js` POST handler:
 - Validates service + city (via `resolveCityFromLocality()` in `lib/slug.js`)
 - Builds slug (`buildSlug()`)
 - Checks slug isn't taken (DB + filesystem of the static-site repo)
 - If admin, validates `post_as` is a valid user
 - Calls `processBeforeAfter()` and `processExtras()` (in `lib/photos.js`) to resize photos with sharp + heic-convert into `tmp/staged/<slug>/<slug>-before.webp` etc.
 - Calls `collectExtras()` to pack service-specific form fields into the extras JSON column
 - Calls `insertDraft()` to write the row to SQLite with `status='draft'`, `submitted_by` = the right user
4. Server calls `generateNarrative()` in `lib/narrative.js` — sends the bullet notes + challenges + voice rules to Claude.
5. Server calls `generateFaq()` in `lib/faq.js` — generates 4 Q&A pairs.
6. Server writes the narrative + faq back to the row via `updateDraft()`.
7. Redirects to `/projects/:id/preview`.
8. Tech clicks "Save & Send for approval".
9. POST `/projects/:id/save` → `markPending(id)` → `status='pending'`.
10. `notifyNewProject()` (in `lib/telegram.js`) sends a message to the admin Telegram chat.
Skipped if a user with role='admin' is submitting.

13.2 Publish flow

1. Admin opens preview → optional edits → POST `/projects/:id/publish`.
2. `publishProject(project)` in `src/routes/publish.js`:
 - `syncSiteRepo()` — `git fetch + reset --hard origin/master` on the local clone (so we start from a clean state).
 - `buildView()` (`lib/render.js`) assembles a view object with every project field plus JSON-LD graph and helper data (nearby cities, related projects, video metadata, etc.).
 - `renderProjectHtml(view)` runs the Handlebars template (`templates/project-page.hbs`) and produces final HTML.
 - Writes `projects/<slug>.html` and copies the staged `.webp` files into `projects/img/`.
 - `updateSpokePage(spokeSlug, project, view)` patches the matching spoke page using cheerio.
 - `writeAffectedArchives(project)` regenerates the master, per-service, and per-spoke archive pages.
 - `updateSitemap(project, action='upsert')` patches `sitemap.xml`.
 - `commitAndPush()` (`lib/git.js`) runs `git add + commit + push`, with `rebase-on-reject + exponential-backoff` retry.
 - `markPublished(id)`.

13.3 YouTube upload flow

1. User picks a video on `/videos` → server-side multer saves to `tmp/videos/<random>`.

2. `uploadVideo({ filePath, mimeType, title, description, tags, privacyStatus })` in `lib/youtube.js`:
 - `getAccessToken()` — if no cached access token, refresh using the stored refresh token from the settings table.
 - POST a JSON metadata body to YouTube's resumable upload endpoint.
 - YouTube returns a `Location` header with the upload session URL.
 - PUT the file bytes via `node:https` (not `fetch` — `fetch` forces chunked encoding, which Google's endpoint rejects).
 - YouTube returns the video ID. Construct `https://www.youtube.com/watch?v=<id>`.
3. If linked to a project, call `updateDraft(projectId, { video_url: url })`. If the project is published, call `republishProject()` to re-render and push.
4. `unlinkSync` the temp file.

13.4 Project page generation

`templates/project-page.hbs` is the canonical template. Every project page is built from the same view object. View fields include:

- `seo` — title, description, canonical URL
- `headline` — hero `h1`
- `tech` — { name, photoUrl, bio, hubName }
- `hero` — before/after photos, hub office name
- `metric` — stats bar items
- `trustSignals` — the 4 badges
- `narrative` — paragraph array
- `beforeAfter` — { beforeUrl, afterUrl }
- `gallery` — extra photo URLs
- `video` — { embedUrl, originalUrl, thumbnailUrl }
- `review` — customer review block
- `serviceTags` — Scope of Work tags
- `related` — 3 related projects with thumbnail URLs
- `nearbyCities` — internal link block
- `map` — Office Location `iframe` URL
- `faq` — Q&A array
- `schema` — the full JSON-LD @graph ready to be JSON-stringified

Edit `templates/project-page.hbs` to change the page's HTML and CSS. Edit `lib/render.js` to change the data that gets passed in.

13.5 Database schema

`projects` table: - `id` PK - `slug` UNIQUE NOT NULL - `service`, `service_label` - `city_slug`, `city_name`, `hub`, `spoke_slug` - `address`, `street`, `address_lat`, `address_lng` - `home_type`, `metric_value`, `metric_label`, `price`, `challenge` - `review_text`, `customer_name`, `review_date`, `review_url`, `customer_note` - `narrative`, `before_photo`, `after_photo` - `extras` (JSON), `extra_photos` (JSON) - `bullet_facts`, `video_url`, `faq` (JSON) - `submitted_by` (FK `users.id`) - `status` (`draft` / `pending` / `published`) - `created_at`, `published_at`

`users`: - `id` PK, `username` UNIQUE, `display_name`, `role`, `password_hash`, `photo_filename`, `bio`, `created_at`, `updated_at`

`settings`: - `key` PK, `value`, `updated_at` Holds YouTube refresh token, per-hub review URLs.

14. Customization points (per-company)

A condensed checklist of every per-company customization. If you're white-labeling this, work through each:

File	What to change
<code>.env</code>	Every value: passwords, API keys, repo paths, hub URLs
<code>src/lib/slug.js</code>	SERVICES array — adapt to the company's services
<code>src/views/partials/details-*.handlebars</code> <code>src/routes/projects.js</code>	One per service form DETAILS_PARTIALS, CHALLENGE_CHOICES, BULLET_PLACEHOLDER, collectExtras
<code>src/lib/render.js</code>	SITE_ROOT, TRUST_SIGNALS, scope-tag builders
<code>src/lib/narrative.js</code>	Voice prompt + example narrative
<code>src/lib/faq.js</code>	Company name + phone fallback
<code>src/lib/video-meta.js</code>	Company name + phone fallback
<code>src/lib/social-post.js</code>	Company name + brand voice
<code>scripts/build-cities.js</code>	HUBS object — every office location
<code>templates/project-page.hbs</code>	Brand colors, logo, footer copy
<code>public/manifest.json</code>	App name + theme color
<code>public/icon-*.png, apple-touch-icon.png</code>	Brand logo
<code>src/views/layouts/main.handlebars</code>	Header logo, navigation copy
<code>src/views/login.handlebars</code>	Login page branding

After all these are done, run `npm run build-cities` once to regenerate `data/cities.json` for the company.

15. Common build-time gotchas

“REQUEST_DENIED” on the new-project address autocomplete

You're using the same key for both the browser-side autocomplete and the server-side geocoding script. Geocoding API doesn't accept HTTP referrer restrictions. Create two separate keys with separate restrictions (section 5.2 step 3).

“fatal: detected dubious ownership in repository”

git refuses to operate on a repo owned by a different user. Add it to `safe.directory`:

```
git config --global --add safe.directory /srv/companyname-site
```

“non-fast-forward” push from the dashboard

Two writers touched the static-site repo. The codebase already handles this with a rebase-on-reject loop (`lib/git.js`). If you see this in `journalctl`, check that no humans are pushing to the same branch from elsewhere.

YouTube uploads land on the wrong channel

Happens when the OAuth flow's brand-account chooser doesn't appear (Google sometimes skips it for repeat auths). Fix: revoke the app at <https://myaccount.google.com/permissions> → reconnect from /videos → the chooser will reappear → carefully pick the company brand.

Cannot GET /<route> after deploying new code

The git pull succeeded but the systemd service wasn't restarted (or pm2 isn't installed and the user thought it was). Always:

```
git pull && sudo systemctl restart companyname-dashboard
```

413 Request Entity Too Large on video upload

nginx is rejecting the upload before it hits Node. Default `client_max_body_size` is 1MB. Raise to 2G:

```
client_max_body_size 2G;
proxy_request_buffering off;
proxy_send_timeout    3600s;
proxy_read_timeout    3600s;
```

Telegram returns “chat not found”

The bot isn't in the group, or the group has no messages yet (bot can only see history after at least one human message), or the chat ID is wrong (group IDs are negative, often start with -100).

Backup script can't push

GitHub deploy keys can't be on more than one repo each. If you're reusing a key, GitHub silently rejects. Create a dedicated key for the backup repo and configure the SSH alias correctly.

Photos rotated sideways on iPhone

EXIF orientation handling. The codebase uses `sharp({ failOn: 'none' }).rotate()` which respects EXIF. If photos still show rotated, the EXIF orientation tag is missing — fix is to add an explicit rotate call in `lib/photos.js`.

Bootstrap admin not created on first launch

`DASHBOARD_PASSWORD` not set in `.env`. The bootstrap only fires when the users table is empty AND `DASHBOARD_PASSWORD` is set. Set it, restart, log in, then unset it for security.

The published page is missing the office-map iframe

Either `gpbEmbedUrl` in `HUBS` is blank for that hub, OR `GOOGLE_MAPS_API_KEY` isn't set so the fallback Maps Embed isn't authenticated. Set one of the two.

This manual gives you everything you need to spin up the dashboard for a new company. For day-to-day operator instructions, see the **User Manual** (*docs/USER-MANUAL.md*).